

Bluetooth printer integration

Tea at Bros · Augmented Human Lab · tea.ahlab.org

Hard constraints

- 1. Android Chrome only.** Web Bluetooth is unsupported on every iOS browser (Safari, Chrome, Firefox — all WebKit, Apple blocks it). iPhone users in the group cannot print from this web app under any circumstance. Don't buy a printer expecting them to.
- 2. BLE (Bluetooth Low Energy), not Bluetooth Classic / SPP.** Web Bluetooth speaks only BLE GATT. Most older "POS portable" thermal printers (including the Custom My Printer we previously evaluated) use Bluetooth Classic SPP — those can't talk to Chrome and are out.
- 3. ESC/POS preferred.** The universal thermal-printer command set. Models using proprietary protocols (NIIMBOT, JINGCHEN B-series) are workable but require reverse-engineered libraries that drift with firmware updates — avoid for the first build.
- 4. HTTPS required.** Web Bluetooth refuses to run on `http://` or `file://` origins. Already covered by tea.ahlab.org via GitHub Pages.

Shortlist (Singapore-purchasable, BLE + ESC/POS)

MODEL	TYPE	PAPER	PRICE (SGD)	NOTES
Goojp ^{rt} PT-210 first-buy	Receipt	58 mm thermal roll	~30–50	Most-documented hobby ESC/POS BLE printer; plenty of community examples driving it from Chrome.
Goojp ^{rt} PT-280	Receipt	58 mm thermal roll, bigger battery	~50–70	Same protocol as PT-210, sturdier build.
MUNBYN ITPP068BT	Receipt / label	58 mm	~60–90	ESC/POS, BLE. Active community.
Phomemo M02 Pro	Label / sticker	53 mm sticker roll	~70–100	Popular pocket sticker. ESC/POS-ish with quirks; community JS libraries exist. Not roll paper.
MUNBYN ITPP047P	Receipt	80 mm thermal	~120–180	Wider receipt format; ESC/POS BLE.
Star Micronics SM-L200	Receipt	58 mm	~300–400	Pro-grade, MFi/BLE, very reliable. Overkill for staff use.
Epson TM-P20 (BT)	Receipt	58 mm	~600–800	Pro-grade. Same comment.

Recommended first buy: Goojprt PT-210. Cheapest, well-documented, has working browser examples from other people's projects, and 58 mm receipt format matches the "print the bill" need. If it doesn't work out the loss is ~SGD 30.

Models to avoid for this project

- Any printer labelled "Bluetooth printer" with no explicit BLE mention — likely Classic SPP, will not pair with Chrome.
- NIIMBOT D110 / D11 / D101 — popular but proprietary protocol; reverse-engineered libraries are brittle.
- Custom My Printer (the one previously evaluated) — Classic SPP only, confirmed.
- Brother / HP / Canon "mobile printers" — these are MFi / AirPrint / Mopria, driven via the OS print stack, not Web Bluetooth.

Wiring outline — future implementation

This is a sketch only, not implemented yet. Approximate scope: 200–300 lines of JS in `index.html`, plus a small persisted "Selected printer" setting.

1. Detect Web Bluetooth support

Feature-detect at boot: `'bluetooth' in navigator`. If false (iOS, older browsers), hide the Print button entirely so nothing appears broken.

2. Add a Print button on the Bill tab

Sit it next to the receipt or in the appbar of the Bill view. Visible only when Web Bluetooth is available. First press launches the pairing flow.

3. Request the device and grab the write characteristic

Open the system Bluetooth picker via `navigator.bluetooth.requestDevice(...)` filtered to the printer's service UUID (Goojprt and most cheap BLE thermal printers use a vendor-common `0000ff00-0000-1000-8000-00805f9b34fb` service with a write characteristic at `0000ff02-...`). User picks the printer from the OS dialog. Chrome remembers the choice across sessions, so this happens once per device.

4. Render the bill as ESC/POS bytes

Build a `Uint8Array` sequence of ESC/POS commands: initialise printer, set alignment / bold / size, write header line ("Tea at Bros"), loop the order lines, draw a separator, write total, advance and cut. The line items already exist in JS (the Bill tab reads them from `counts` and `MENU`).

5. Stream the bytes to the printer

BLE has a per-write MTU around 20 bytes (often 100+ on newer phones, but assume 20 for safety). Split the byte array into chunks and write sequentially to the characteristic with a short delay between chunks. Wrap in try/catch and surface a clear error if the connection drops.

6. Persist the chosen printer (optional)

Once a printer is paired, store its device-id hint in `localStorage` so the next visit skips the picker if Chrome still remembers it. If Chrome has forgotten (happens after deep app-switches / power cycles), gracefully re-request.

7. Fallback for non-Android users

Add a "Print via your phone's printer" button that calls `window.print()` with a print stylesheet that hides everything except the receipt section. Routes through AirPrint / Mopria / any cloud printer the user already has set up. Not as nice as a 58 mm receipt printer for this use case, but the only path on iOS.

Risks and practical caveats

- **Per-vendor ESC/POS quirks.** Every cheap thermal printer ignores some subset of standard ESC/POS commands and substitutes its own. Goojprt PT-210's quirks are well-documented; budget half a day to tune the byte sequence after the first prints come out mis-aligned.
- **Pairing memory drift.** Android occasionally forgets paired Web Bluetooth devices between Chrome restarts or after several days. Expect a "select printer" tap every few sessions; don't promise users a permanent connection.
- **Battery and paper.** Mobile thermal printers are small Li-ion + thermal heads. Keep it charged, buy 2–3 spare rolls, store rolls flat.
- **Thermal print fade.** Receipts printed on thermal paper fade over months of exposure to sunlight or heat. Fine for daily-use snack bills; not fine for accounting records.
- **No iPhone path without native app.** Repeat: iOS users cannot print from this web app via Bluetooth, ever. AirPrint via `window.print()` is the only iOS path, and only works with AirPrint-certified printers (most cheap thermal ones are not).

One-line summary: buy a Goojprt PT-210 (~SGD 30–50), implement Web Bluetooth + ESC/POS in ~250 lines, accept that only Android users will print and iPhones must use AirPrint or skip the feature.

Source: `/Users/sankha/Projects/ahlab-org/ahl-tea-html5/docs/printer-integration.html` .
regenerate the PDF with: `"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" --headless=new --print-to-pdf=docs/printer-integration.pdf docs/printer-integration.html`